Práctica 6: La Inteligencia Colectiva (Modelos de Ensambles)

Objetivo: Comprender por qué combinar múltiples modelos puede mejorar la capacidad de generalización frente a un clasificador individual. En particular, se estudiará el fenómeno del *overfitting* en árboles de decisión, el funcionamiento de estrategias de ensamble como **Random Forest** y **Boosting**, y la forma en que estos métodos logran reducir errores, estabilizar predicciones e identificar variables relevantes en un problema de clasificación real.

Material de partida

Se proporciona lo siguiente:

- Un dataset real de clasificación, **Breast Cancer Wisconsin**, integrado en `scikit-learn`, por lo que no es necesario descargar ningún fichero externo.
- Una plantilla de código base en Python con `scikit-learn` y `matplotlib`, para facilitar la carga de datos, el entrenamiento de los modelos y la representación gráfica de resultados.
- Un problema de clasificación binaria con múltiples variables predictoras, apropiado para observar tanto el sobreajuste de un árbol individual como las ventajas de los métodos de ensamble.

Nota

En esta práctica se proporcionará una implementación base para acelerar la parte mecánica del trabajo. El objetivo no es que construyas desde cero los algoritmos de ensamble, sino que entiendas qué problema intentan resolver, cómo se comportan frente a un árbol simple y qué información puede extraerse de sus resultados.

Introducción

En muchos problemas reales, una única decisión tomada por un único modelo puede ser frágil. Un árbol de decisión sin restricciones, por ejemplo, puede adaptarse con enorme precisión a los datos de entrenamiento, pero esa misma flexibilidad puede convertirse en una debilidad cuando el modelo se enfrenta a ejemplos nuevos. Este fenómeno se conoce como **sobreajuste** u *overfitting*, y constituye uno de los problemas centrales del aprendizaje automático.
Una estrategia clásica para reducir este problema consiste en combinar múltiples modelos en lugar de confiar en uno solo. Esta idea da lugar a los **métodos de ensamble**, donde varios clasificadores cooperan para producir una predicción final más robusta. En lugar de depender de un único árbol potencialmente inestable, se construye un conjunto de árboles que votan, corrigen errores o reparten la carga de aprendizaje.
En esta práctica vas a trabajar con tres enfoques progresivos:

- un **árbol de decisión simple**, que servirá como línea base;
- un **Random Forest**, que combina múltiples árboles entrenados sobre distintas muestras de los datos;
- y un modelo de **Boosting**, que entrena árboles secuencialmente para corregir errores previos.

El objetivo es que comprendas no solo cuál obtiene mejor precisión, sino también **por qué** se comportan de forma distinta y qué compromisos existen entre capacidad predictiva, coste de entrenamiento, interpretabilidad y robustez.

Tarea 1: El árbol solitario

En esta primera parte vas a entrenar un árbol de decisión sin limitar su complejidad, con el fin de utilizarlo como referencia inicial.

Qué debes hacer

- Carga el dataset **Breast Cancer Wisconsin** proporcionado por `scikit-learn`.
- Divide los datos en entrenamiento y prueba.
- Entrena un modelo `DecisionTreeClassifier` sin imponer restricciones de profundidad.
- Calcula el rendimiento del modelo tanto en entrenamiento como en prueba.
- Compara ambos resultados.

Cuestión

Debes explicar por qué un árbol de decisión muy profundo puede alcanzar resultados excelentes en entrenamiento y, sin embargo, empeorar al evaluar sobre datos no vistos.
No basta con indicar que el modelo «memoriza». Debes razonar qué significa eso en términos de particiones del espacio de entrada, sensibilidad al ruido y capacidad de generalización.

Análisis

Comenta si observas un comportamiento compatible con *overfitting*. En particular, debes interpretar una situación en la que el árbol obtenga una precisión muy alta, o incluso perfecta, en entrenamiento, pero claramente inferior en test.
El objetivo de esta parte es que entiendas que una alta precisión en entrenamiento no garantiza un buen modelo.

Tarea 2: Bagging y Random Forest

Ahora vas a estudiar cómo mejora el comportamiento del árbol cuando, en lugar de usar uno solo, se combinan muchos árboles entrenados sobre muestras diferentes de los datos.

Qué debes hacer

- Entrena un modelo `RandomForestClassifier`.
- Repite el experimento variando el número de árboles, por ejemplo con valores como `1`, `10`, `50` y `100`.
- Evalúa el rendimiento en el conjunto de prueba para cada configuración.
- Representa en una gráfica la evolución del rendimiento frente al número de árboles.

Qué debes interpretar

Debes explicar por qué Random Forest suele generalizar mejor que un árbol individual.
Tu explicación debe relacionar, al menos de forma intuitiva, ideas como:

- el entrenamiento de múltiples árboles sobre subconjuntos distintos de ejemplos;
- la reducción de la varianza del modelo;
- y el efecto de la votación agregada sobre errores individuales.

Cuestión

Analiza si llega un momento en el que añadir más árboles deja de mejorar claramente el resultado.
No se trata solo de indicar que la curva se estabiliza. Debes explicar por qué, a partir de cierto punto, el ensemble ya ha capturado casi toda la mejora que puede obtenerse con esta estrategia.

Tarea 3: El poder del Boosting

En esta parte trabajarás con una estrategia distinta. Mientras que Random Forest construye árboles en paralelo para promediar decisiones, el Boosting entrena árboles de manera secuencial, intentando corregir en cada paso los errores cometidos por los anteriores.

Qué debes hacer

- Entrena un modelo de `GradientBoostingClassifier` o, alternativamente, de `AdaBoostClassifier`.
- Evalúa su rendimiento sobre el conjunto de prueba.
- Mide o compara su tiempo de entrenamiento respecto a Random Forest.
- Extrae la información asociada a la importancia de variables del modelo entrenado.

Interpretación

Debes explicar en qué se diferencia conceptualmente el Boosting del Bagging.
En particular, debes comentar que en Boosting los modelos no son independientes entre sí, sino que cada nuevo árbol se construye para concentrarse en los errores o residuos que aún no han sido bien modelados.

Análisis

A partir de `feature_importances_`, identifica las **3 variables** que el modelo considera más relevantes y comenta si te parece razonable que esas características influyan en la clasificación.
No es necesario realizar una interpretación médica exhaustiva del dataset, pero sí debes mostrar que entiendes que el modelo no trata todas las variables por igual y que algunas aportan más capacidad discriminativa que otras.

Tarea 4: Comparación global de los modelos

Una vez entrenados los tres enfoques, vas a compararlos de manera conjunta.

Qué debes hacer

Debes elaborar una comparación entre:

- el **árbol simple**;
- el **Random Forest**;
- y el modelo de **Boosting**.

La comparación debe incluir, como mínimo:

- rendimiento en test;
- comportamiento frente al sobreajuste;
- coste de entrenamiento aproximado;
- y facilidad de interpretación.

Cuestión

Debes responder razonadamente a esta idea central:
¿Por qué un conjunto de modelos puede ser mejor que un único modelo aparentemente muy potente?
Tu respuesta debe mostrar que entiendes el papel de la diversidad, la agregación de decisiones y la reducción del error de generalización.

Reflexión

Comenta también qué inconvenientes presentan los métodos de ensamble. Por ejemplo, puedes discutir aspectos como:

- el mayor coste computacional;
- la pérdida de interpretabilidad frente a un árbol individual;
- o la dificultad de desplegar modelos grandes en entornos limitados.

Reto: Pensando en el despliegue

Imagina ahora que debes integrar uno de estos modelos en una aplicación con recursos limitados, como un dispositivo móvil o un sistema con poca memoria y CPU.

Qué debes hacer

Debes elegir cuál de los tres modelos desplegarías en ese contexto:

- árbol simple;
- Random Forest;
- o Boosting.

Cuestiones

Debes responder razonadamente a estas preguntas:

- ¿Qué modelo elegirías?
- ¿Qué ganarías con esa decisión?
- ¿Qué estarías sacrificando a cambio?

Interpretación

El objetivo de esta parte es que entiendas que elegir un modelo no depende solo de la precisión. En problemas reales también importan el coste computacional, la latencia, la memoria disponible y la facilidad de mantenimiento.

Formato de entrega

Debes entregar un **Informe de Resultados (PDF)** que incluya:

- una breve descripción del dataset utilizado y del problema de clasificación planteado;
- los resultados del árbol de decisión en entrenamiento y prueba, junto con su interpretación;
- la gráfica de evolución del rendimiento de Random Forest frente al número de árboles;
- la comparación entre Random Forest y Boosting en términos de rendimiento y tiempo de entrenamiento;
- la identificación de las 3 variables más importantes según el modelo de Boosting;
- una tabla comparativa final entre árbol simple, Random Forest y Boosting;
- y una conclusión razonada sobre qué modelo elegirías en un escenario real y por qué.

Recomendaciones

A la hora de redactar el informe, evita limitarte a copiar salidas automáticas o fragmentos de código sin explicarlos. Debes interpretar los resultados obtenidos y relacionarlos con las ideas teóricas trabajadas en la práctica.
Conviene que prestes especial atención a cuestiones como:

- por qué un árbol puede sobreajustar con facilidad;
- por qué Random Forest tiende a ser más estable;
- en qué sentido Boosting corrige errores de forma secuencial;
- qué relación existe entre rendimiento, coste computacional e interpretabilidad;
- y por qué una mejor precisión no siempre implica que un modelo sea la mejor elección para desplegarlo.

El informe debe ser claro, sintético y razonado. Deben priorizarse los resultados, las evidencias experimentales y su interpretación.

Objetivo final de la práctica

El objetivo final de esta práctica es que entiendas que los métodos de ensamble constituyen una de las ideas más potentes del aprendizaje automático moderno: combinar modelos relativamente simples para obtener predicciones más robustas y fiables. Al mismo tiempo, debes comprender que estas mejoras no son gratuitas, ya que suelen implicar un mayor coste computacional y una menor interpretabilidad que los modelos individuales.